Introduction to Ensemble Learning

Contents

1. Overview . . . 2
2. The wisdom of the crowd . . . 3
3. How an ensemble predicts . . . 4
    3.1 The base models must be different . . . 5
    3.2 Classification: majority vote . . . 6
    3.3 Regression: the mean . . . 6
4. The four types of ensemble . . . 7
    4.1 Voting . . . 8
    4.2 Stacking . . . 8
    4.3 Bagging . . . 9
    4.4 Boosting . . . 9
5. Why ensembles work . . . 10
6. Costs and benefits . . . 11
    6.1 The cost: more computation . . . 11
    6.2 Benefit 1: better performance . . . 12
    6.3 Benefit 2: lower variance or lower bias . . . 12
    6.4 Benefit 3: robustness . . . 13
7. When to use an ensemble . . . 13
8. Summary . . . 14
9. Sources . . . 14
10. Key terms . . . 15

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Bias-variance trade-off; Decision trees.
- **Leads to:** Voting ensembles; Bagging; Boosting; Stacking and blending.

1. Overview

Key point: An ensemble trains several different models and combines their answers: a **majority vote** (G-1146) for classification, a mean for regression. The combined model is usually better than any one of them.

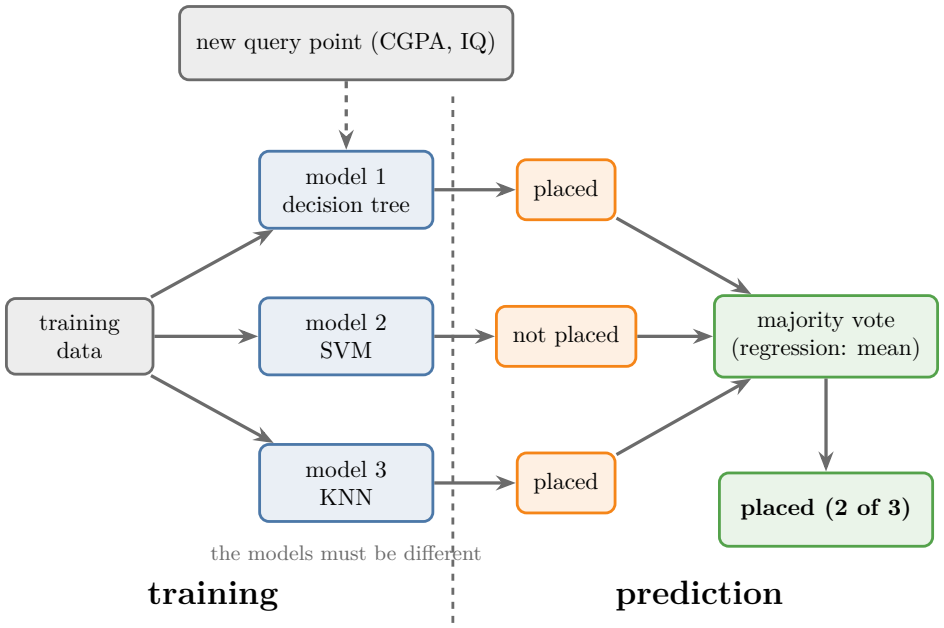


Figure 1: An ensemble: different models trained on the data, their predictions combined into one

The word **ensemble** means a group, for example a group of musicians. **Ensemble learning** (G-689) (first met as a step of the ML development life cycle) combines several ML models into one bigger, stronger model (Figure 1).

Ensembles are tried in almost every serious ML project and in almost every Kaggle competition (Kaggle is a website that hosts data-science contests). This Note covers:

- the idea behind ensembles, the **wisdom of the crowd** (G-2124) (section 2);
- how an ensemble predicts, and why its models must differ (section 3);

- the four main types: voting, stacking, **bagging** (G-251) and boosting, each with its own Notes later (section 4);
- why ensembles work (section 5), what they cost and what they give (section 6), and when to use them (section 7).

2. The wisdom of the crowd

> **Key point:** The combined judgement of many people is often more accurate than the judgement of any one of them. **Ensemble learning** (G-689) applies the same idea to models.

The **wisdom of the crowd** (G-2124) is the observation that a crowd, taken together, often knows the answer better than a single member. We rely on it every day:

- **Audience poll in a quiz show.** In *Kaun Banega Crorepati* (*Who Wants to Be a Millionaire*), a contestant can ask the studio audience. The most popular answer is right most of the time, although no single person in the audience is an expert in everything.
- **Product reviews.** A product rated 4.5 out of 5 by 15,000 buyers feels safe to buy. The same 4.5 from one buyer does not.
- **Film ratings.** An IMDb rating such as 8.5 is the combined opinion of many viewers.
- **Democracy.** Everyone votes, and the party with the most votes wins, because we trust what most people support.
- **Guessing a weight.** At a fair, visitors guessed the weight of an animal on show. Nobody guessed it exactly, but the average of all the guesses was very close to the true weight.

> **Extra:** The weight-guessing story is real. At a fair in Plymouth, England, in 1906, about 800 visitors guessed the weight of an ox. Francis Galton reported that the middle guess (the median) was 1,207 pounds, within 1% of the true 1,198 pounds (Galton 1907). The book *The Wisdom of Crowds* collects many such cases (Surowiecki 2004).

Figure 2 shows the mechanism on simulated guesses of that weight. Each guess is the true weight, 1,198 pounds, plus a random error of about 75 pounds, sometimes too high and sometimes too low. Step by step:

1. **One guess** is typically off by 60 pounds.
2. **Average 25 guesses.** The guesses that are too high and the guesses that are too low partly cancel. The mean is typically off by 12 pounds.
3. **Average 800 guesses.** The cancelling is almost complete. The mean is typically off by 2 pounds.

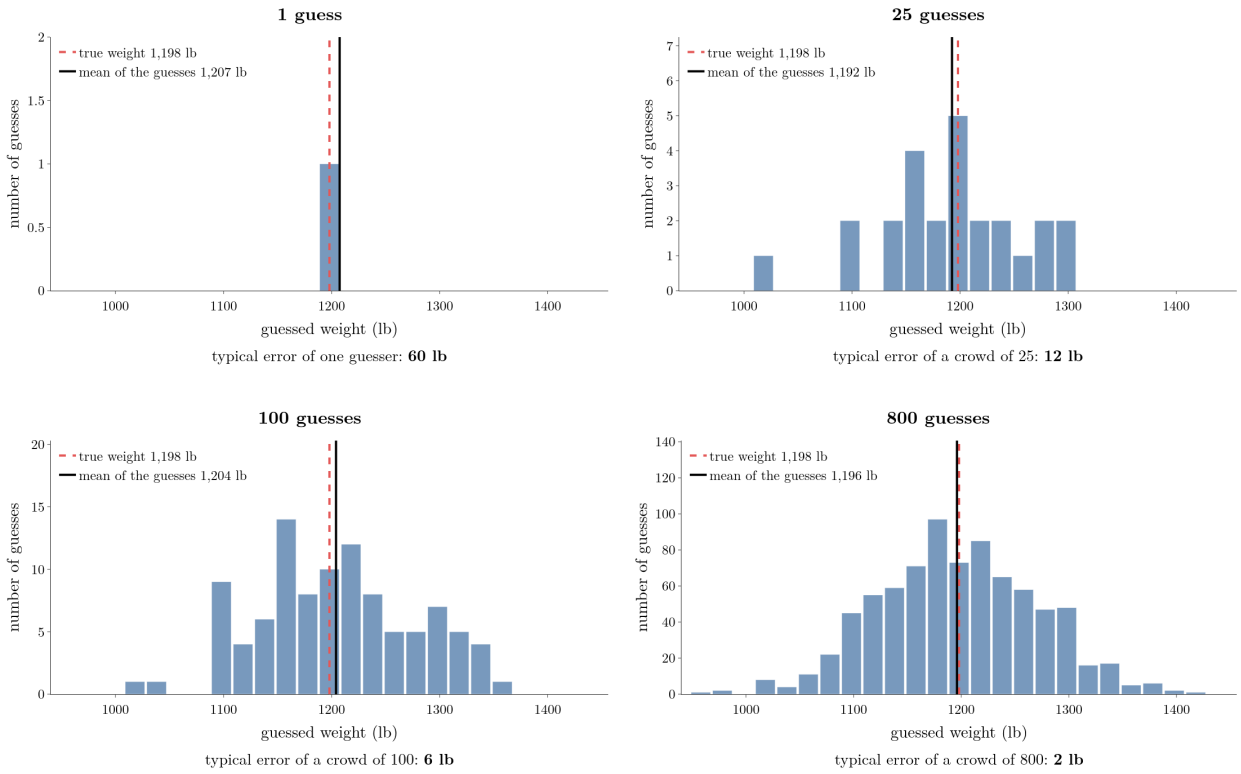


Figure 2: Simulated guesses of an ox's weight (true weight 1,198 lb, each guess off by about 75 lb at random). As the crowd grows from 1 to 800, the mean of the guesses (black) closes in on the true weight (red dashed). The typical error is averaged over 2,000 simulated crowds of each size

In Figure 2, watch the black line: the histogram of single guesses stays wide, but its mean moves onto the red line. The errors must be different from guess to guess for the cancelling to happen; a crowd in which everyone makes the same mistake gains nothing from averaging.

Ensemble learning rests on the same fact: a crowd of models knows more than one model.

3. How an ensemble predicts

> **Key point:** Every base model predicts; the ensemble takes the **majority vote** (G-1146) (classification) or the mean (regression).

Every ML algorithm has two stages: **training**, where it finds the pattern in the data, and **prediction**, where it answers for a new query point (a new **observation**, one record or row of the data table, as when KNN predicts). Each observation has **features** (input variables, one column each) and a **target** (the output we predict). Training differs between the types of ensemble (section 4). Prediction works the same way for all of them.

An ensemble is a collection of smaller models, called **base models** (G-260). They can be any algorithms: **decision trees** (G-561) (models that predict by asking yes/no questions),

SVMs (models that separate classes with the widest possible margin), KNN, linear regression and so on.

3.1 The base models must be different

> **Key point:** A crowd only helps if its members know different things. We make models different by using different algorithms, different data, or both.

Picture the quiz-show audience again. If everyone in it is a software developer, the poll helps with a computer science question and fails on everything else. If the audience mixes an accountant, an engineer, a doctor, a politician and a philosopher, someone is likely to know the answer whatever the topic.

Models are the same: an ensemble of identical models makes the same mistakes as one model. There are three ways to make the **base models** (G-260) different:

1. **Different algorithms, same data:** for example a linear model, an SVM and a **decision tree** (G-561), all trained on the full dataset.
2. **Same algorithm, different data:** for example three linear models, each trained on a different part of the data. Different data makes them learn differently.
3. **Both:** different algorithms, each shown different data.

In Figure 3 and in Figure 6a, each dot is a data point with two features (its position), coloured by its true class, and the pale background colour shows what the vote predicts at that position: orange area, one class; blue area, the other. Where the colour changes is the decision boundary. A dot sitting in the wrong colour is a point the vote gets wrong. Figure 3 tests the first way on the two-moons data (two interlocking half-moon classes, used again in section 5). Watch the left panel: three copies of the same tree draw one **decision boundary** (G-555), the line where the prediction switches class, so their vote is that tree and scores the same 0.875. On the right, three different algorithms disagree in places, and their vote scores 0.890.

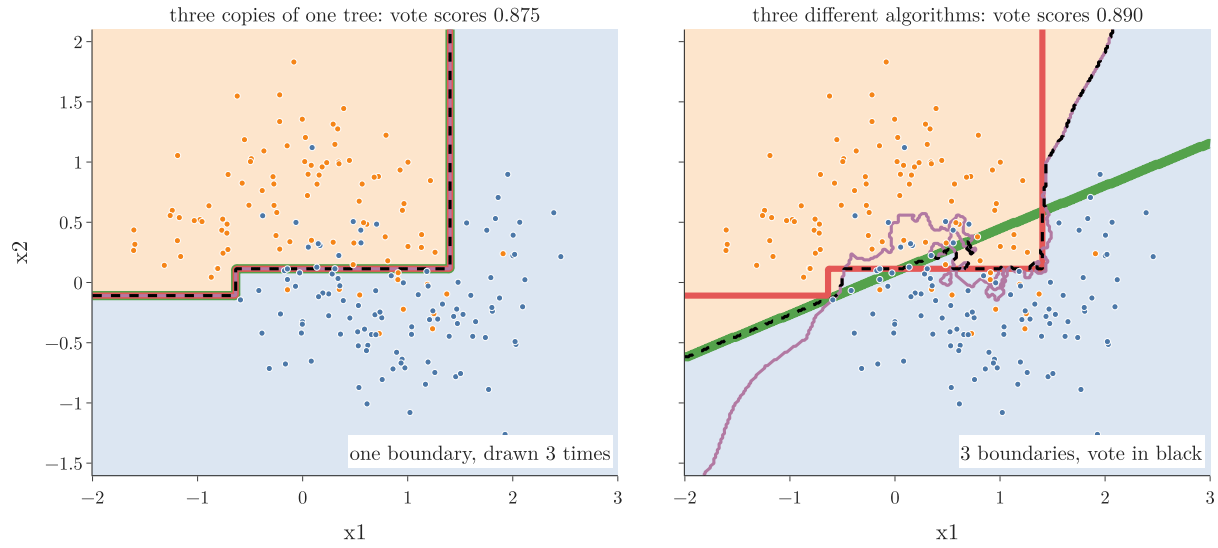


Figure 3: Left: three copies of one depth-3 tree, trained on the same data, and their vote. Right: logistic regression (green), a depth-3 tree (red) and KNN with k = 5 (purple), and their vote (black dashed). Test accuracy of the vote in each title

3.2 Classification: majority vote

> **Key point:** Each model votes for a class, and the class with the most votes wins.

Take the placement data: from a student's CGPA and IQ, predict *placed* or *not placed* (as in the placement toy project). We have 5 trained **base models** (G-260) and one new student.

We give the student's CGPA and IQ to every model. Suppose 3 models say *placed* and 2 say *not placed*. The ensemble answers *placed*: the **majority vote** (G-1146), exactly as KNN votes among neighbours. Figure 4 draws this vote with 5 models; Figure 1 shows the same with 3.

3.3 Regression: the mean

> **Key point:** Each model predicts a number, and the ensemble returns their average.

Now the task is to predict the package, in lakh rupees per year (LPA), from CGPA and IQ. Every **base model** (G-260) is a regression model and outputs a number.

1. **In words:** the ensemble's prediction is the mean of the base models' predictions.
2. **Formula:** with n base models predicting $\hat{y}_1, \ldots, \hat{y}_n$,

$$\hat{y} = \frac{1}{n} \sum_{i=1}^{n} \hat{y}_i$$

3. **Example:** three models predict 6.2, 5.8 and 7.0 LPA:

$$\hat{y} = \frac{6.2 + 5.8 + 7.0}{3} = \frac{19.0}{3} = 6.33 \text{ LPA}$$

Figure 4 draws both worked examples, the vote of section 3.2 and the mean of section 3.3. Watch the vote take the class most models chose, and the mean land between the three predictions on the number line.

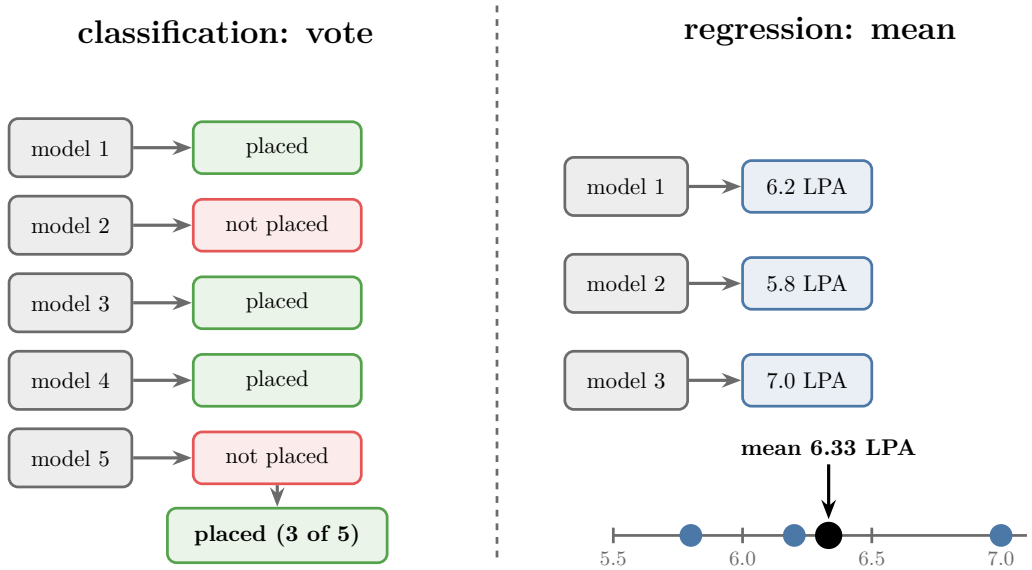


Figure 4: Left: five models vote, 3 for placed and 2 for not placed, so the ensemble says placed. Right: three models predict 6.2, 5.8 and 7.0 LPA, and the ensemble returns their mean, 6.33 LPA

4. The four types of ensemble

> **Key point:** Voting and stacking use different algorithms on the same data; **bagging** (G-251) and boosting use one algorithm on varied data. They differ in how the models are trained and combined.

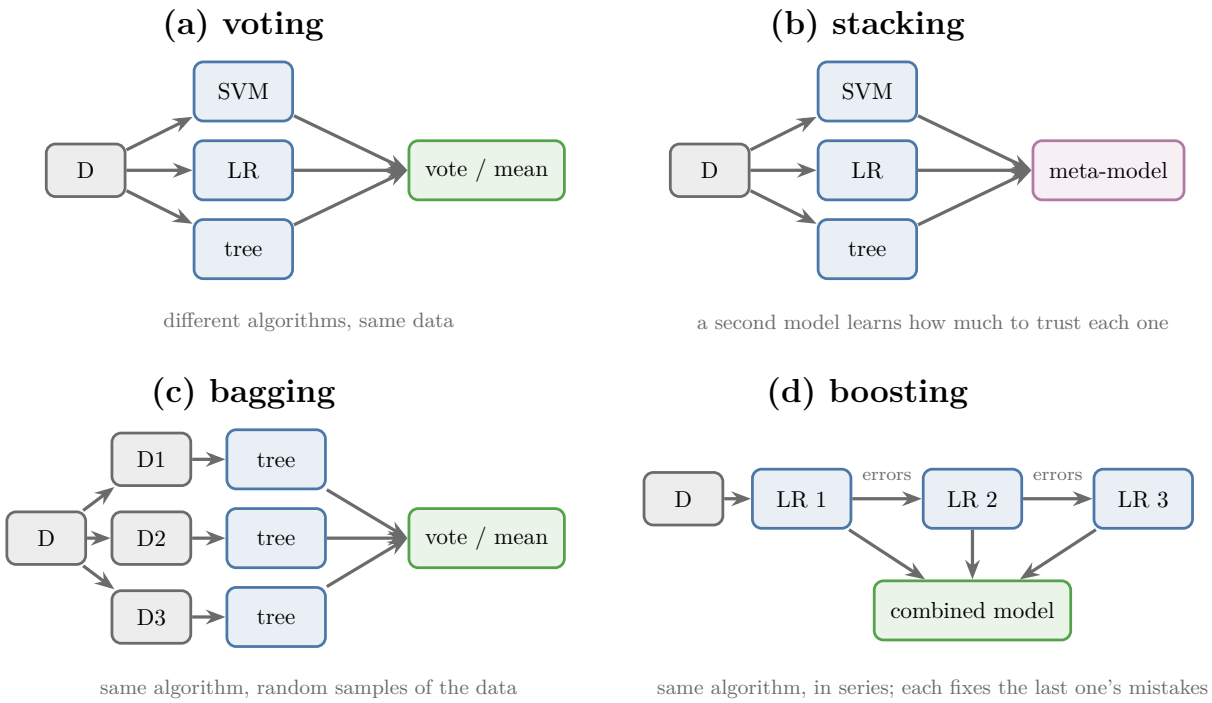


Figure 5: The four main types of ensemble

Figure 5 shows the four types. Each one is taught in full in later Notes; here we only meet them.

4.1 Voting

> **Key point:** Different algorithms, all trained on the same data; the final answer is the **majority vote** (G-1146) or the mean.

Take, say, an SVM, a logistic regression and a **decision tree** (G-561), and train all three on the same dataset D. For a new query point, each predicts, and we take the majority (classification) or the mean (regression). The variety comes from the algorithms being different. Voting is the subject of the next three Notes, starting with the **voting ensemble** (G-2096).

4.2 Stacking

> **Key point:** Voting plus one more model on top, which learns how much weight each **base model** (G-260)'s opinion deserves.

Stacking starts like voting: different algorithms trained on the same data. Then a further model, the **meta-model** (G-1213) (for example KNN), is trained on the base models' outputs.

Each training observation gives the meta-model one example: what the SVM said, what the logistic regression said, what the tree said, and what the true answer was. For instance, (1, 0,

1) with true answer 1, or (0, 0, 1) with true answer 1.

From such observations the meta-model learns a weight for each base model: more weight to the models that are usually right, less to the ones that often err. In voting every model's vote counts the same, as in a democracy. In stacking, some votes count more than others. Stacking comes last in the ensemble Notes.

4.3 Bagging

> **Key point:** One algorithm, many copies, each trained on a different random sample of the observations; the answers are then voted or averaged.

Bagging (G-251) is short for **bootstrap aggregation**. All **base models** (G-260) use the same algorithm, for example three SVMs or three logistic regressions. The variety comes from the data.

Suppose the data D has 1,000 students and we decide to show each model 500 of them. We draw 500 observations at random, with replacement (each drawn observation is put back, so the same one can be drawn twice), to make D1 and train model 1 on it. Then we draw another 500 the same way for D2, and so on. Drawing random samples like this is called **bootstrapping** (G-322).

The samples differ, so the models learn differently. At prediction time we vote or average as before.

When the base models are **decision trees** (G-561), the bagging ensemble gets its own name: a **random forest** (G-1611), a "forest" of trees. Strictly, a random forest also picks a random subset of the features at every split (Breiman 2001); see how the base model differs. Later we cover the core idea of bagging and how a random forest works.

4.4 Boosting

> **Key point:** Models are trained one after another; each one concentrates on the mistakes of the one before.

Boosting (G-318) also uses one algorithm, but the models are trained **in series**, not side by side. The first model is trained on the data and notes which observations it got wrong. The next model is told about those observations and pays extra attention to them, fixing some of them.

The second model then passes its own mistakes to the third model, and so on. Down the chain the mistakes shrink, and the combined model makes far fewer than the first. Boosting is the most powerful of the four, and also the most complex; it has several Notes of its own, starting with AdaBoost.

5. Why ensembles work

> **Key point:** Each model makes its own random mistakes; combining them cancels much of that noise, leaving a smoother, more reliable answer.

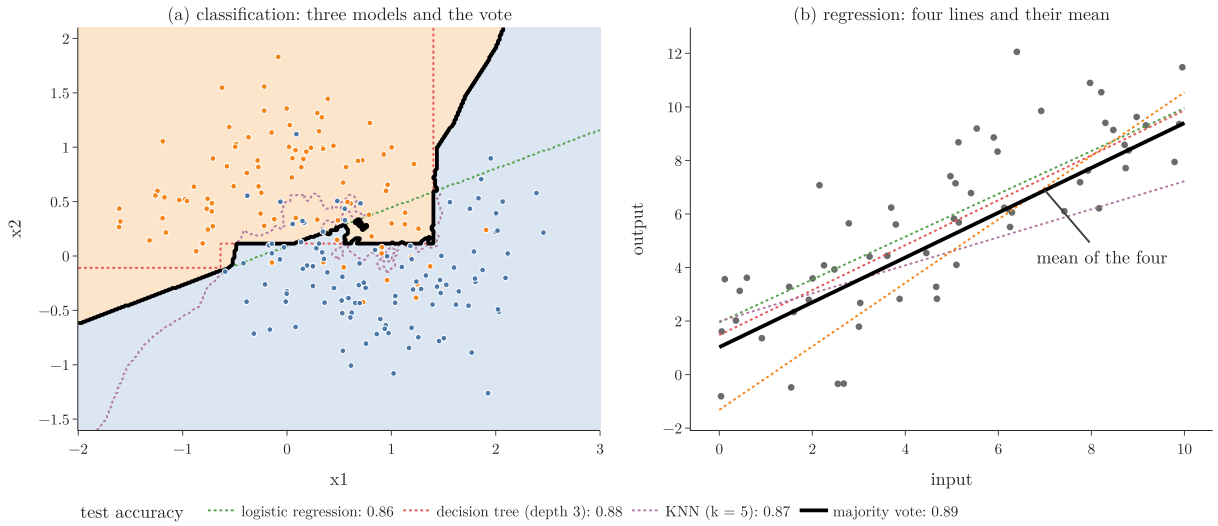


Figure 6: (a) Three classifiers' decision boundaries (dotted) and their majority-vote decision boundary (black) on two-moons data; (b) four lines, each fitted to 10 random points, and their mean

Classification. In Figure 6a, three different models are trained on the same two-moons data (two interlocking half-moon classes):

- **logistic regression** draws a straight line;
- a **decision tree** of depth 3 draws boxes;
- **KNN** with k = 5 draws a wiggly curve.

Each decision boundary is wrong in its own places. The **majority vote** (G-1146) (black) follows a decision boundary where at least two of the three agree, so the quirks of any single model are outvoted. On 200 test points, the three models score 0.860, 0.875 and 0.870; the vote scores **0.890**, better than each.

Regression. In Figure 6b, four lines are fitted, each to a different set of 10 random points. One is too steep, one too flat, two are in between. Their mean (black) is not extreme in any direction: it lands in the middle, close to the real trend.

Figure 7 keeps adding lines, each fitted to its own 10 random points, up to 25. Watch the black mean settle near the true trend (red dashed) while single lines keep scattering. Averaged over 500 such crowds, one line misses the true trend by 0.75 (root mean squared gap), the mean of 5 lines by 0.45 and the mean of 25 by 0.37.

The gain slows down after a few lines because every line is fitted to points drawn from the

same 60 noisy observations. The noise in those 60 points is an error the lines share, and averaging cannot cancel a shared error (section 6.3). One line fitted to all 60 points misses the true trend by 0.34, and the mean of many lines closes in on that floor, not on 0.

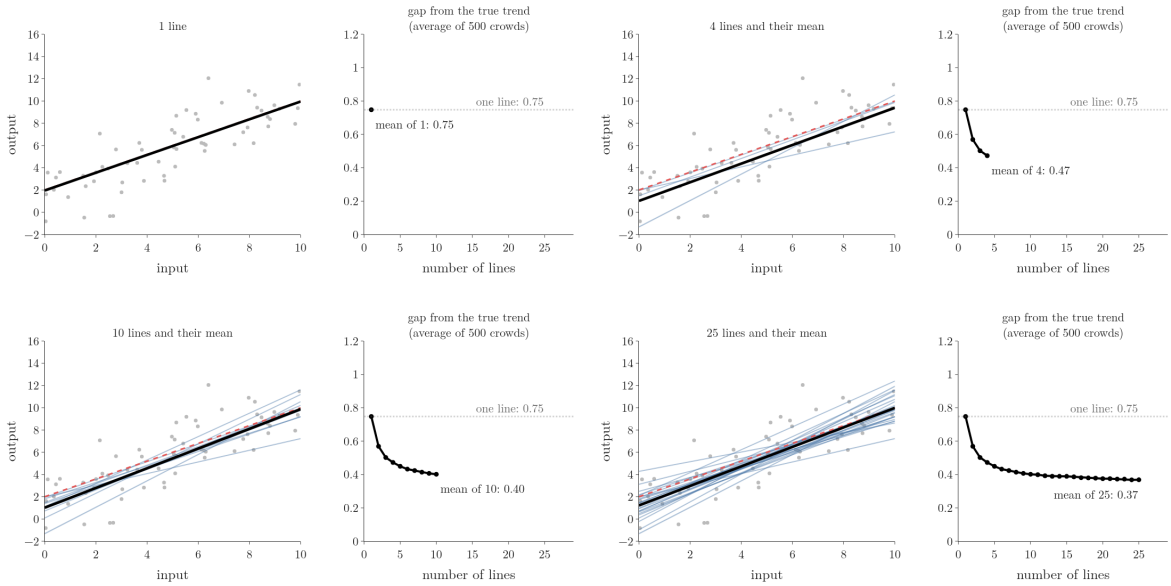


Figure 7: Lines added one at a time, each fitted to 10 random points of the same data. Left: the lines (blue), their mean (black) and the true trend (red dashed). Right: the gap between the mean line and the true trend, averaged over 500 crowds

The vote does not always beat the best model; hard and soft voting shows a case where it does not. Why and when combining helps is proved in why voting works: the probability.

6. Costs and benefits

> **Key point:** An ensemble costs more computation. In return it usually gives better performance, lower **variance** (G-2073) (voting and bagging) or lower bias (boosting), and more robust results.

6.1 The cost: more computation

> **Key point:** Training many models takes longer than training one.

Where we used to train one model, we now train many: tens, hundreds, even thousands. Training and prediction take longer. So an ensemble needs a solid advantage to be worth it, and it has three.

In Figure 8, across is the number of trees; the time axis on the left is a log scale (equal distances mean equal multiplication: the labelled ticks 10, 30, 100 and 300 are each about 3 times the one before), and the right panel shows accuracy on test data that the trees did not train on. Figure 8 puts the cost next to the first benefit, for **bagging** (G-251) ensembles

of 1 to 200 full-depth trees on the two-moons data of section 5. Watch training time keep climbing with the number of trees, while test accuracy rises from 0.83 to about 0.87 by 20 trees and then levels off, above the 0.84 of one full tree.

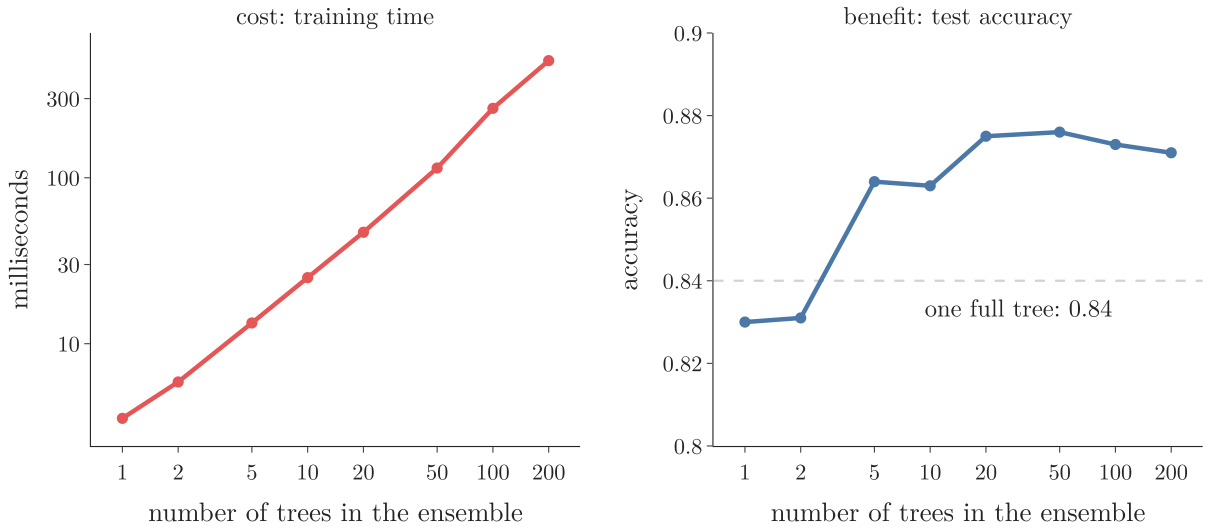


Figure 8: Bagging ensembles of 1 to 200 full-depth trees on the two-moons data. Left: training time (log scale). Right: test accuracy, averaged over 5 seeds; dashed: one full tree

6.2 Benefit 1: better performance

> **Key point:** The combined model usually scores better than its **base models** (G-260).

As section 5 showed, combining models usually improves accuracy (classification) or R^2 (regression).

6.3 Benefit 2: lower variance or lower bias

> **Key point:** Averaging models (voting, **bagging** (G-251)) lowers variance and leaves bias unchanged. Training models in series (boosting) lowers bias. Either way the ensemble reaches low bias and low variance together, which a single model rarely can.

A single model usually trades **bias** (G-287) against **variance** (G-2073) (the **bias-variance trade-off** (G-288)). Ensembles escape the trade-off in two different ways:

- **Bagging and voting lower variance.** They average models. Averaging cancels the errors that differ from model to model, as in Figure 2, and that part of the error is the variance. Averaging cannot remove an error that all the models share: if every model is too low by 2 on average, their mean is also too low by 2. So the bias stays the same. Bagging therefore starts from low-bias, high-variance models such as deep trees and cuts their variance (see why bagging works).

- **Boosting lowers bias.** Each new model corrects the mistakes the earlier ones still make. **Boosting** (G-318) therefore starts from simple high-bias models and cuts their bias (see parallel against sequential learning).

In short: bagging reduces variance, boosting reduces bias.

6.4 Benefit 3: robustness

> **Key point:** The ensemble's performance changes little when the data changes.

A **robust** model (**robustness**, G-1700) keeps performing well when the data it sees changes somewhat. Robustness is closely related to low **variance** (G-2073), but it is often listed as a benefit in its own right.

7. When to use an ensemble

> **Key point:** Almost always: try one at the end of every project, after cleaning, feature engineering, training and evaluating single models.

There is rarely a reason not to try an ensemble. In a project, it usually comes last: after data cleaning, preprocessing, feature engineering, model building and evaluation, we combine models and check whether the result improves. Most of the time it does.

Ensembles made Kaggle competitions famous, and Kaggle in turn made XGBoost, a **boosting** (G-318) algorithm, famous: in 2015, 17 of the 29 winning solutions published on Kaggle's blog used XGBoost (Chen and Guestrin 2016). On small and medium-sized tables of data, tree ensembles often beat deep learning; on very large datasets, deep learning can take the lead. A benchmark on 45 medium-sized tables found tree-based models still ahead of deep networks (Grinsztajn et al. 2022).

The order of the coming Notes: voting, then bagging, then **random forests** (G-1611), then boosting, and stacking last.

8. Summary

Type	Base models	Data each model sees	Combined by	Mainly reduces
Voting	different algorithms	the same data	majority vote or mean	variance (errors that differ between models)
Stacking	different algorithms	the same data	a meta-model that learns weights	errors of individual models
Bagging	one algorithm (random forest: trees)	random samples of the observations	majority vote or mean	variance
Boosting	one algorithm, in series	observations weighted toward earlier mistakes	weighted combination	bias

- An ensemble combines several base models: **majority vote** (G-1146) for classification, mean for regression, because errors that go in different directions partly cancel, as the crowd's mean guess closes in on the ox's true weight.
- The **base models** (G-260) must differ: different algorithms, different data, or both, because identical models make the same mistakes and their vote is just one model (0.875 for three copies of one tree).
- Ensembles cost more computation but usually improve performance and are more robust, so an ensemble is worth trying at the end of almost every project. Voting and bagging lower **variance** (G-2073), because averaging cancels the errors that differ between models but not an error they share; boosting lowers bias, because each new model corrects the mistakes the earlier ones still make.
- On two-moons data, three models scoring 0.860 to 0.875 give a vote scoring 0.890, because the quirks of any single boundary are outvoted by the other two; this is why a crowd of different models usually beats any one of them.

9. Sources

Built from

- CampusX, "Introduction to Ensemble Learning | Ensemble Techniques in Machine Learning", YouTube, https://www.youtube.com/watch?v=bHK1fE_BUms

Other references

- **Breiman 2001:** L. Breiman, "Random Forests", *Machine Learning* 45, 5–32, 2001.
- **Chen and Guestrin 2016:** T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System", *Proceedings of KDD 2016*, 785–794. arxiv.org/abs/1603.02754
- **Galton 1907:** F. Galton, "Vox Populi", *Nature* 75, 450–451, 1907.

- **Grinsztajn et al. 2022:** L. Grinsztajn, E. Oyallon and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?", NeurIPS 2022 Datasets and Benchmarks Track.
- **Surowiecki 2004:** J. Surowiecki, *The Wisdom of Crowds*, Doubleday, 2004.

10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Wisdom of the crowd (G-2124)	The combined judgement of many is often more accurate than any one member's.
Bagging (bootstrap aggregation) (G-251)	Training many models on different random samples of the data and averaging them, so the result depends less on the particular sample (lower variance).
Base model (G-260)	One of the models inside an ensemble; the ensemble combines the base models' answers into its prediction.
Meta-model (G-1213)	The final model in stacking: it takes the other models' (base models') predictions as its inputs and is trained to combine them.
Bootstrapping (G-322)	Drawing random samples of the data to train each base model.
Boosting (G-318)	Combining many simple models one after another to reduce bias, the error of a model that is too simple.
Robustness (G-1700)	Performing well even when the data changes somewhat.
Ensemble (G-690)	Several trained models whose predictions are combined.
Majority vote (G-1146)	Predicting the class that most voters choose: the k neighbours in KNN, or the base models of an ensemble.
Ensemble learning (G-689)	Combining several models into one stronger model.
Decision tree (G-561)	A model that predicts by asking a chain of questions about the input columns: nested if-else conditions.
Decision boundary (G-555)	A line or curve that separates the classes in classification.

Term	Meaning
Voting ensemble (G-2096)	Several models trained on the same data, combined by majority vote (classification) or mean (regression).
Random forest (G-1611)	Bagging with decision trees as the base models: many trees, each trained on a random sample of the rows with a random choice of features at every split, vote (classification) or are averaged (regression), which lowers the variance of a single tree.
Variance (of a model) (G-2073)	How much a model's predictions change when it is trained on a different sample of the data; a different meaning from the variance of a column.
Bias (of a model) (G-287)	Error from a model being too simple to capture the true relationship.
Bias-variance trade-off (G-288)	Lowering bias by adding complexity tends to raise variance, and the reverse.